

1. Write a python code to apply data cleaning using the Delete Rows & Columns method.

Ans: import pandas as pd

Load dataset

```
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")
```

----- Delete Columns -----

Remove unwanted columns (example: Area and Email)

```
df = df.drop(['Area', 'Email'], axis=1, errors='ignore')
```

----- Delete Rows -----

Remove rows that contain any missing values

```
df = df.dropna()
```

Show cleaned data

```
print(df.head())
```

```
print("\nRemaining Columns:", df.columns)
```

Output:

	ExamNumber	FirstName	LastName	Gender	Weight	Pincode	Mobile	Salary
0	20084341011	Yash	Patel	Male	79	382721	9714999161	10000
1	20084341014	Darshan	Rajput	Male	57	382480	9537531531	25000
2	20084341002	Shashank	Jain	Male	87	380008	9099918110	10000
3	20084341004	Manoj	Nai	Male	74	382424	9904676228	55000
4	20084341005	Akshay	Patel	Male	57	382330	9724243538	45000

```
Remaining Columns: Index(['ExamNumber', 'FirstName', 'LastName', 'Gender', 'Weight', 'Pincode',
                          'Mobile', 'Salary'],
                          dtype='object')
```

2. Write a python code to apply data cleaning using Missing value imputation by Mean, Median method.

Ans: import pandas as pd

Load dataset

```
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")
```

```
# ----- Mean Imputation -----

# Example: fill missing values in numeric columns using Mean

df['Weight'] = df['Weight'].fillna(df['Weight'].mean())

df['Salary'] = df['Salary'].fillna(df['Salary'].mean())

# ----- Median Imputation -----

# Example: fill missing values using Median

df['Pincode'] = df['Pincode'].fillna(df['Pincode'].median())

# Show updated dataset

print(df.head())
```

Output:

	ExamNumber	FirstName	LastName	Gender	Weight	Area	Pincode	\
0	20084341011	Yash	Patel	Male	79	Kalol	382721	
1	20084341014	Darshan	Rajput	Male	57	Ranip	382480	
2	20084341002	Shashank	Jain	Male	87	Maninagar	380008	
3	20084341004	Manoj	Nai	Male	74	Chandkeda	382424	
4	20084341005	Akshay	Patel	Male	57	Nana Chiloda	382330	

	Mobile	Email	Salary
0	9714999161	patelyash605@gmail.com	10000
1	9537531531	darshanrajput89@gmail.com	25000
2	9099918110	jsashank11@gmail.com	10000
3	9904676228	manojnai56@gmail.com	55000
4	9724243538	akshu.p20800@gmail.com	45000

3. Write a python code to apply data cleaning using Numerical Missing Value Imputation By Class.

Ans: import pandas as pd

```
# Load dataset
```

```
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")
```

```
# ----- Numerical Missing Value Imputation By Class -----
```

```
# Example: Impute 'Weight' column based on 'Gender' class
```

```
df['Weight'] = df.groupby('Gender')['Weight'].transform(
    lambda x: x.fillna(x.mean())
```

```
)

# Example 2: Impute 'Salary' based on 'Area' class

df['Salary'] = df.groupby('Area')['Salary'].transform(

    lambda x: x.fillna(x.median())

)

# Show output

print(df.head())
```

Output:

	ExamNumber	FirstName	LastName	Gender	Weight	Area	Pincode	\
0	20084341011	Yash	Patel	Male	79	Kalol	382721	
1	20084341014	Darshan	Rajput	Male	57	Ranip	382480	
2	20084341002	Shashank	Jain	Male	87	Maninagar	380008	
3	20084341004	Manoj	Nai	Male	74	Chandkeda	382424	
4	20084341005	Akshay	Patel	Male	57	Nana Chiloda	382330	

	Mobile	Email	Salary
0	9714999161	patelyash605@gmail.com	10000
1	9537531531	darshanrajput89@gmail.com	25000
2	9099918110	jsashashank11@gmail.com	10000
3	9904676228	manojnai56@gmail.com	55000
4	9724243538	akshu.p20800@gmail.com	45000

4. Write a python code to apply data cleaning using Missing Value Imputation - Categorical Variable.

Ans: import pandas as pd

Load dataset

```
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")
```

----- Categorical Missing Value Imputation -----

Example: Fill missing values in Gender column using Mode

```
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])
```

Example: Fill missing values in Area column using Mode

```
df['Area'] = df['Area'].fillna(df['Area'].mode()[0])
```

```
# Example: Fill missing values in FirstName or LastName (if needed)
```

```
df['FirstName'] = df['FirstName'].fillna(df['FirstName'].mode()[0])
```

```
df['LastName'] = df['LastName'].fillna(df['LastName'].mode()[0])
```

```
# Show updated dataset
```

```
print(df.head())
```

Output:

	ExamNumber	FirstName	LastName	Gender	Weight	Area	Pincode	\
0	20084341011	Yash	Patel	Male	79	Kalol	382721	
1	20084341014	Darshan	Rajput	Male	57	Ranip	382480	
2	20084341002	Shashank	Jain	Male	87	Maninagar	380008	
3	20084341004	Manoj	Nai	Male	74	Chandkeda	382424	
4	20084341005	Akshay	Patel	Male	57	Nana Chiloda	382330	

	Mobile	Email	Salary
0	9714999161	patelyash6050@gmail.com	10000
1	9537531531	darshanrajput89@gmail.com	25000
2	9099918110	jsashashank11@gmail.com	10000
3	9904676228	manojnai56@gmail.com	55000
4	9724243538	akshu.p20800@gmail.com	45000

5. Write a python code to apply data cleaning using Missing value imputation using Scikit-Learn for Numeric and Categorical Variables/Data.

Ans:

```
import pandas as pd
```

```
from sklearn.impute import SimpleImputer
```

```
# Load dataset
```

```
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")
```

```
# ----- Separate Numeric & Categorical Columns -----
```

```
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
```

```
categorical_cols = df.select_dtypes(include=['object']).columns
```

```
# ----- Numeric Imputation (Mean) -----
```

```
num_imputer = SimpleImputer(strategy='mean')
```

```

df[numeric_cols] = num_imputer.fit_transform(df[numeric_cols])

# ----- Categorical Imputation (Most Frequent) -----

cat_imputer = SimpleImputer(strategy='most_frequent')

df[categorical_cols] = cat_imputer.fit_transform(df[categorical_cols])

# Show output

print(df.head())

print("\nNumeric Columns:", numeric_cols)

print("Categorical Columns:", categorical_cols)

```

Output:

	ExamNumber	FirstName	LastName	Gender	Weight	Area	Pincode	\
0	2.008434e+10	Yash	Patel	Male	79.0	Kalol	382721.0	
1	2.008434e+10	Darshan	Rajput	Male	57.0	Ranip	382480.0	
2	2.008434e+10	Shashank	Jain	Male	87.0	Maninagar	380008.0	
3	2.008434e+10	Manoj	Nai	Male	74.0	Chandkeda	382424.0	
4	2.008434e+10	Akshay	Patel	Male	57.0	Nana Chiloda	382330.0	

	Mobile	Email	Salary
0	9.714999e+09	patelyash605@gmail.com	10000.0
1	9.537532e+09	darshanrajput89@gmail.com	25000.0
2	9.099918e+09	jsshshank11@gmail.com	10000.0
3	9.904676e+09	manojnai56@gmail.com	55000.0
4	9.724244e+09	akshu.p20800@gmail.com	45000.0


```

Numeric Columns: Index(['ExamNumber', 'Weight', 'Pincode', 'Mobile', 'Salary'], dtype='object')
Categorical Columns: Index(['FirstName', 'LastName', 'Gender', 'Area', 'Email'], dtype='object')

```

6. Write a python code to apply data preprocessing using One hot encoding?

Ans:

```

import pandas as pd

# Load dataset

df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")

# ----- One Hot Encoding -----

# Example: Encode Gender, Area columns

df_encoded = pd.get_dummies(df, columns=['Gender', 'Area'], drop_first=False)

```

```
print(df_encoded.head())
```

Output:

```
ExamNumber  FirstName  LastName  Weight  Pincode  Mobile  \
0  20084341011      Yash      Patel      79   382721  9714999161
1  20084341014  Darshan  Rajput      57   382480  9537531531
2  20084341002  Shashank      Jain      87   380008  9099918110
3  20084341004     Manoj      Nai      74   382424  9904676228
4  20084341005   Akshay      Patel      57   382330  9724243538

Email  Salary  Gender_Female  Gender_Male  ...  \
0  patelyash605@gmail.com    10000          False          True  ...
1  darshanrajput89@gmail.com    25000          False          True  ...
2  jsshshank11@gmail.com    10000          False          True  ...
3  manojnai56@gmail.com     55000          False          True  ...
4  akshu.p20800@gmail.com    45000          False          True  ...

Area_Nana  Chiloda  Area_Naroda  Area_Navavadaaj  Area_Nirnaynagar  \
0          False          False          False          False
1          False          False          False          False
2          False          False          False          False
3          False          False          False          False
4           True          False          False          False

Area_Ranip  Area_Sahibuag  Area_Satellite  Area_Science  City  Area_Sola  \
0          False          False          False          False  False
1           True          False          False          False  False
2          False          False          False          False  False
3          False          False          False          False  False
4          False          False          False          False  False

Area_navrangpura
0          False
1          False
2          False
3          False
4          False

[5 rows x 27 columns]
```

7. Write a python code to apply data preprocessing using Label encoding

Ans: import pandas as pd

from sklearn.preprocessing import LabelEncoder

Load dataset

df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")

Create LabelEncoder object

le = LabelEncoder()

----- Label Encode Categorical Columns -----

```
categorical_cols = ['FirstName', 'LastName', 'Gender', 'Area']
```

```
for col in categorical_cols:
```

```
    df[col] = le.fit_transform(df[col])
```

```
print(df.head())
```

Output:

	ExamNumber	FirstName	LastName	Gender	Weight	Area	Pincode	\
0	20084341011	17	8	1	79	4	382721	
1	20084341014	2	9	1	57	11	382480	
2	20084341002	16	3	1	87	5	380008	
3	20084341004	11	7	1	74	0	382424	
4	20084341005	0	8	1	57	7	382330	

	Mobile	Email	Salary
0	9714999161	patelyash605@gmail.com	10000
1	9537531531	darshanrajput89@gmail.com	25000
2	9099918110	jsashashank11@gmail.com	10000
3	9904676228	manojnai56@gmail.com	55000
4	9724243538	akshu.p20800@gmail.com	45000

8. Write a python code to apply preprocessing using Ordinal encoding?

Ans: import pandas as pd

```
from sklearn.preprocessing import OrdinalEncoder
```

```
# Load dataset
```

```
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")
```

```
# ----- Ordinal Encoding -----
```

```
# Example: Suppose we create an ordinal order for 'Area' manually
```

```
# (just an example order, you can modify)
```

```
area_order = [['Nana Chiloda', 'Ranip', 'Kalol', 'Maninagar', 'Chandkeda']]
```

```
encoder = OrdinalEncoder(categories=area_order)
```

```
df['Area_Ordinal'] = encoder.fit_transform(df[['Area']])
```

```
print(df[['Area', 'Area_Ordinal']].head())
```


Output:

	ExamNumber	FirstName	LastName	Gender	Weight	Area	Pincode	\
0	20084341011	17.0	8.0	1.0	79	4.0	382721	
1	20084341014	2.0	9.0	1.0	57	11.0	382480	
2	20084341002	16.0	3.0	1.0	87	5.0	380008	
3	20084341004	11.0	7.0	1.0	74	0.0	382424	
4	20084341005	0.0	8.0	1.0	57	7.0	382330	

	Mobile	Email	Salary
0	9714999161	patelyash605@gmail.com	10000
1	9537531531	darshanrajput89@gmail.com	25000
2	9099918110	jsshashank11@gmail.com	10000
3	9904676228	manojnai56@gmail.com	55000
4	9724243538	akshu.p20800@gmail.com	45000

9. Write a python code to apply feature scaling using Normalization method

Ans:

```
import pandas as pd
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
# Load dataset
```

```
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")
```

```
# Select numeric columns
```

```
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
```

```
# Apply Normalization (Min-Max Scaling)
```

```
scaler = MinMaxScaler()
```

```
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
```

```
print(df.head())
```

```
print("\nNormalized Columns:", numeric_cols)
```


Output:

```
ExamNumber  FirstName  LastName  Gender  Weight  Area  Pincode  \
0          1.0      Yash      Patel   Male   0.705882  Kalol  0.001005
1          1.0    Darshan    Rajput   Male   0.058824  Ranip  0.000916
2          1.0   Shashank      Jain   Male   0.941176  Maninagar  0.000000
3          1.0     Manoj      Nai    Male   0.558824  Chandkeda  0.000895
4          1.0    Akshay     Patel   Male   0.058824  Nana Chiloda  0.000860

Mobile      Email      Salary
0  0.916729  patelyash605@gmail.com  0.000000
1  0.864675  darshanrajput89@gmail.com  0.051724
2  0.736313  jsshashank11@gmail.com  0.000000
3  0.972366  manojnai56@gmail.com  0.155172
4  0.919441  akshu.p20800@gmail.com  0.120690

Normalized Columns: Index(['ExamNumber', 'Weight', 'Pincode', 'Mobile', 'Salary'], dtype='object')
```

10. Write a python code to apply feature scaling using Standardization Method?

Ans: import pandas as pd

from sklearn.preprocessing import StandardScaler

Load dataset

df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")

Select numeric columns

numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns

Apply Standardization (Z-score scaling)

scaler = StandardScaler()

df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

print(df.head())

print("\nStandardized Columns:", numeric_cols)

Output:

```
ExamNumber  FirstName  LastName  Gender  Weight  Area  Pincode  \
0  0.535094      Yash      Patel    Male  0.508955  Kalol -0.240070
1  0.535094    Darshan    Rajput   Male -1.306776  Ranip -0.240460
2  0.535094   Shashank      Jain    Male  1.169221  Maninagar -0.244459
3  0.535094     Manoj      Nai     Male  0.096289  Chandkeda -0.240550
4  0.535094    Akshay     Patel    Male -1.306776  Nana Chiloda -0.240703

Mobile      Email      Salary
0  0.479825  patelyash605@gmail.com -0.590466
1  0.298079  darshanrajput89@gmail.com -0.352161
2 -0.150082  jsshashank11@gmail.com -0.590466
3  0.674074  manojnai56@gmail.com 0.124448
4  0.489292  akshu.p20800@gmail.com -0.034422

Standardized Columns: Index(['ExamNumber', 'Weight', 'Pincode', 'Mobile', 'Salary'], dtype='object')
```

11. Write a python code to apply Simple Linear Regression.

Ans: import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

import matplotlib.pyplot as plt

Load dataset

df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")

----- Select Feature (X) and Target (y) -----

X = df[["Weight"]] # Independent variable

y = df["Salary"] # Dependent variable

----- Split Data -----

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

----- Train Model -----

model = LinearRegression()

model.fit(X_train, y_train)

----- Predictions -----

y_pred = model.predict(X_test)

```
# ----- Output -----

print("Intercept:", model.intercept_)

print("Coefficient:", model.coef_[0])

print("\nPredicted Salary Values:\n", y_pred)

# ----- Visualization -----

plt.scatter(X_test, y_test)

plt.plot(X_test, y_pred)

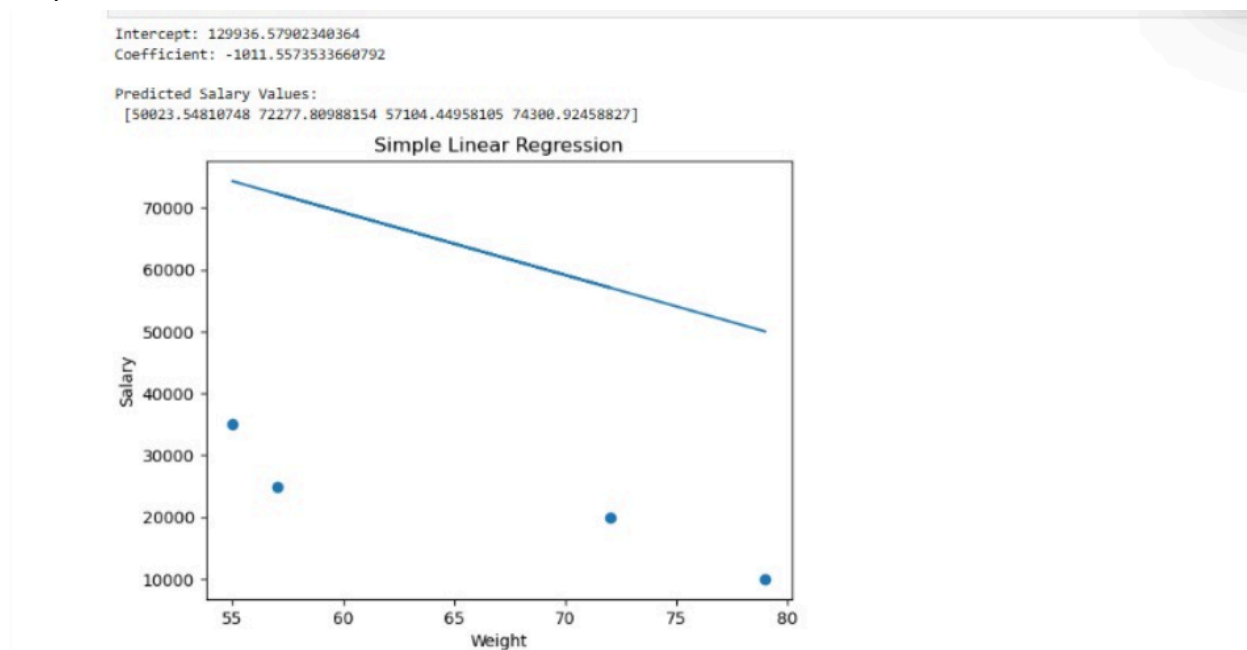
plt.xlabel("Weight")

plt.ylabel("Salary")

plt.title("Simple Linear Regression")

plt.show()
```

Output:



12. Write a python code to apply Multiple Linear Regression?

Ans: import pandas as pd

from sklearn.model_selection import train_test_split

```

from sklearn.linear_model import LinearRegression

from sklearn.preprocessing import LabelEncoder

# Load dataset

df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")

# ----- Encode Categorical Columns -----

le = LabelEncoder()

df['Gender'] = le.fit_transform(df['Gender'])

df['Area'] = le.fit_transform(df['Area'])

# ----- Select Multiple Features (X) and Target (y) -----

X = df[['Weight', 'Pincode', 'Gender', 'Area']] # independent variables

y = df['Salary'] # dependent variable


# ----- Split Data -----

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----- Train Model -----

model = LinearRegression()

model.fit(X_train, y_train)

# ----- Predictions -----

y_pred = model.predict(X_test)

# ----- Output -----

print("Intercept:", model.intercept_)

print("Coefficients:", model.coef_)

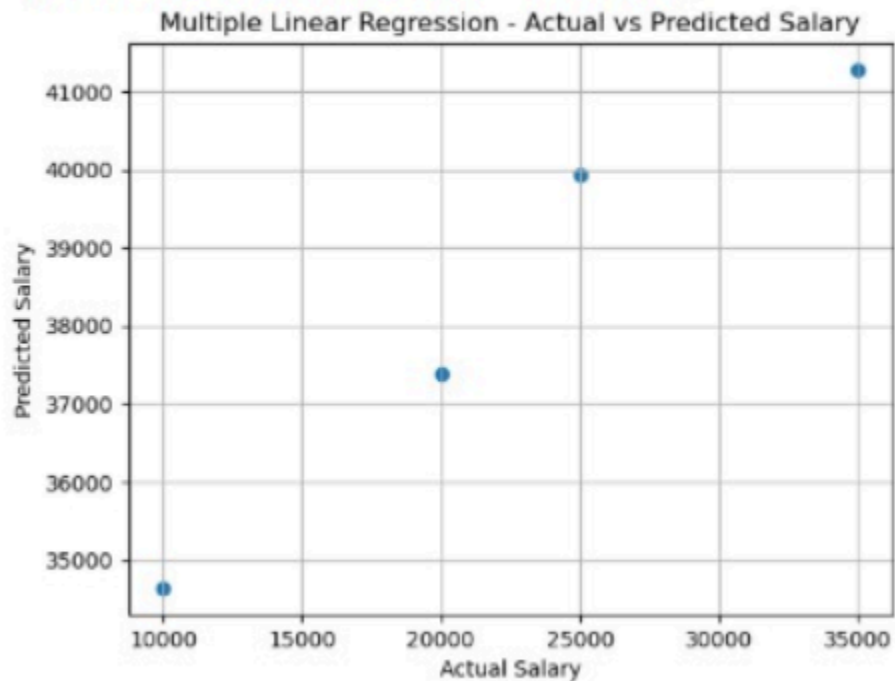
```

```
print("\nPredicted Salary Values:\n", y_pred)
```

Output:

```
Intercept: 210711.50420907675  
Coefficients: [-3.43820275e+02 -5.50292736e-03 -1.45501389e+05 -3.25838990e+02]
```

```
Predicted Salary Values:  
[34638.87176179 39923.37108214 37386.338093 41276.26533294]
```



13. Write a python code to apply Polynomial Regression.

Ans:

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.preprocessing import PolynomialFeatures
```

```
import matplotlib.pyplot as plt
```

```
# Load dataset
```

```

df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")

# ----- Select Feature and Target -----

X = df[['Weight']] # independent variable

y = df['Salary'] # dependent variable

# ----- Polynomial Features (degree 2) -----

poly = PolynomialFeatures(degree=2)

X_poly = poly.fit_transform(X)

# ----- Train-Test Split -----

X_train, X_test, y_train, y_test = train_test_split(
    X_poly, y, test_size=0.2, random_state=42
)

# ----- Train Model -----

model = LinearRegression()

model.fit(X_train, y_train)

# ----- Predictions -----

y_pred = model.predict(X_test)

# ----- Output -----

print("Coefficients:", model.coef_)

print("Intercept:", model.intercept_)

print("\nPredicted Salary:\n", y_pred)

# ----- Visualization -----

# Sort values for smooth curve

X_sorted = np.sort(X.values, axis=0)

X_poly_sorted = poly.transform(X_sorted)

```

```

y_curve = model.predict(X_poly_sorted)

plt.scatter(X, y, color='blue')

plt.plot(X_sorted, y_curve, color='red')

plt.xlabel("Weight")

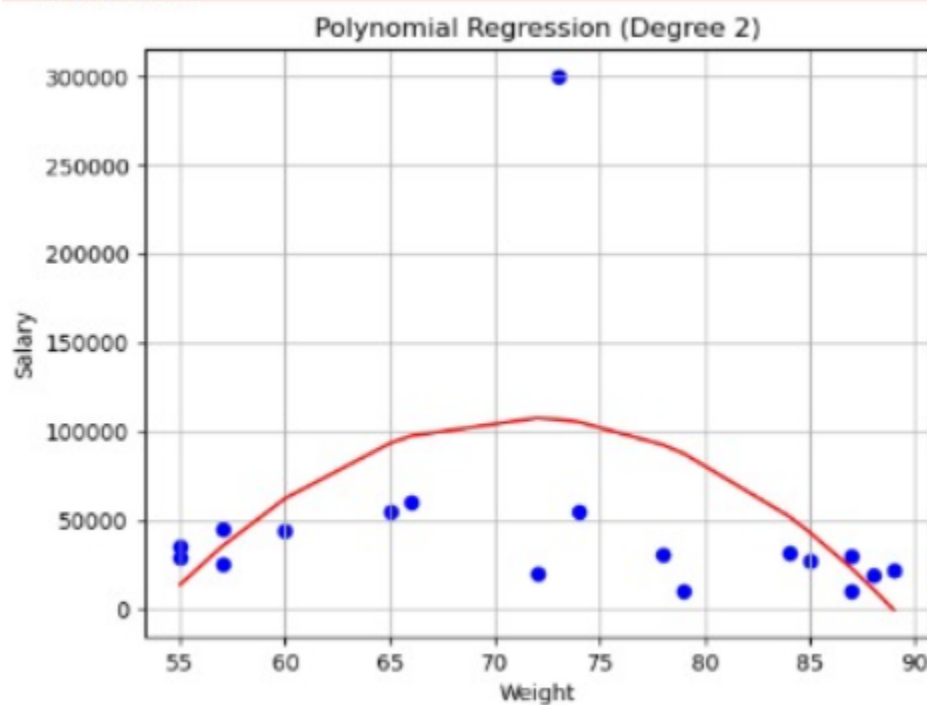
plt.ylabel("Salary")

plt.title("Polynomial Regression (Degree 2)")

plt.grid(True)

plt.show()
Output:

```



14. Write a python code to apply Linear Regression Assumption?

Ans:

```
import pandas as pd
```

```
import numpy as np
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```



```

from sklearn.linear_model import LinearRegression

from sklearn.model_selection import train_test_split

from scipy.stats import shapiro

# Load dataset

df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")

# Select columns

X = df[["Weight"]]

y = df['Salary']

# Split data

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model

model = LinearRegression()

model.fit(X_train, y_train)

# Predictions

y_pred = model.predict(X_test)

# Residuals

residuals = y_test - y_pred

# =====

# 1. Linearity (Actual vs Predicted Plot)

# =====

plt.scatter(y_test, y_pred)

plt.xlabel("Actual Salary")

plt.ylabel("Predicted Salary")

plt.title("Linearity Check: Actual vs Predicted")

```

```

plt.grid(True)

plt.show()

# =====

# 2. Homoscedasticity (Residual Plot)

# =====

plt.scatter(y_pred, residuals)

plt.axhline(0, color='red')

plt.xlabel("Predicted Values")

plt.ylabel("Residuals")

plt.title("Homoscedasticity Check")

plt.grid(True)

plt.show()

# =====

# 3. Normality of Residuals (Histogram + Shapiro Test)

# =====

sns.histplot(residuals, kde=True)

plt.title("Normality of Residuals")

plt.show()

stat, p = shapiro(residuals)

print("\nShapiro-Wilk Test:")

print("Statistic:", stat)

print("p-value:", p)

# =====

# 4. Independence (Residuals vs Index)

```

```
# =====
```

```
plt.plot(residuals)
```

```
plt.title("Independence Check (Residuals Independence)")
```

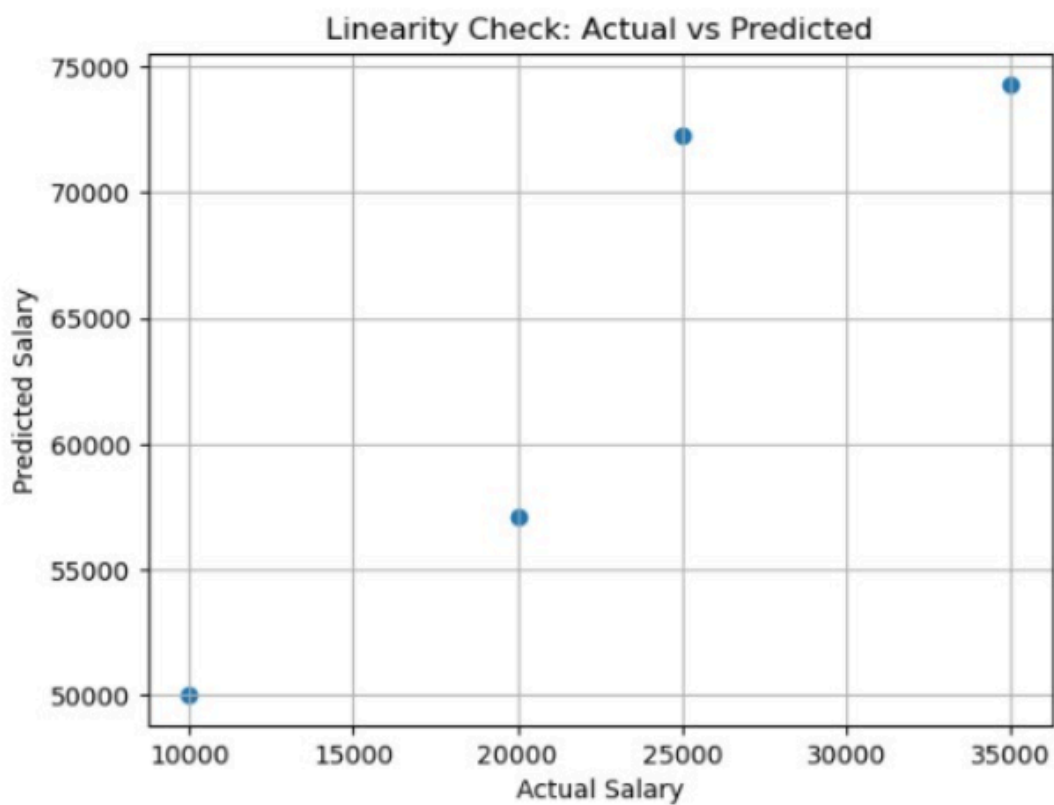
```
plt.xlabel("Index")
```

```
plt.ylabel("Residual")
```

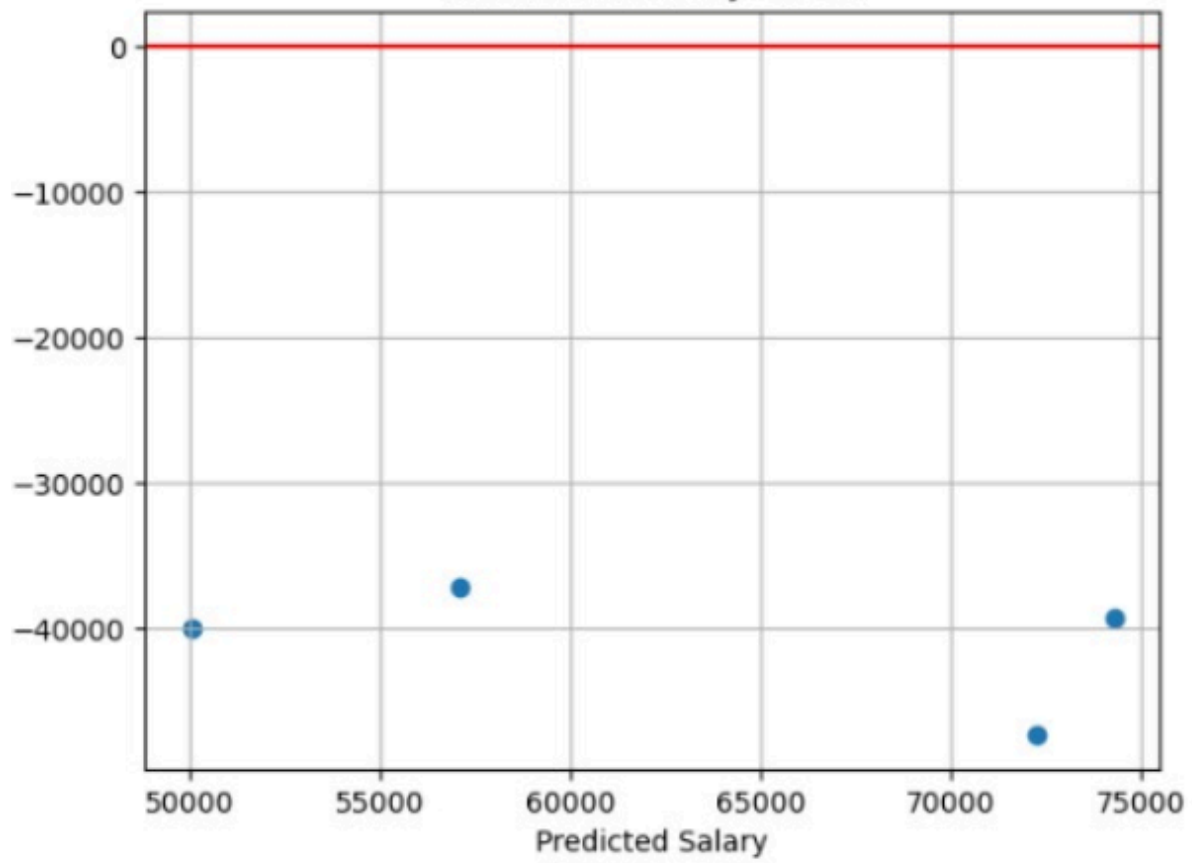
```
plt.grid(True)
```

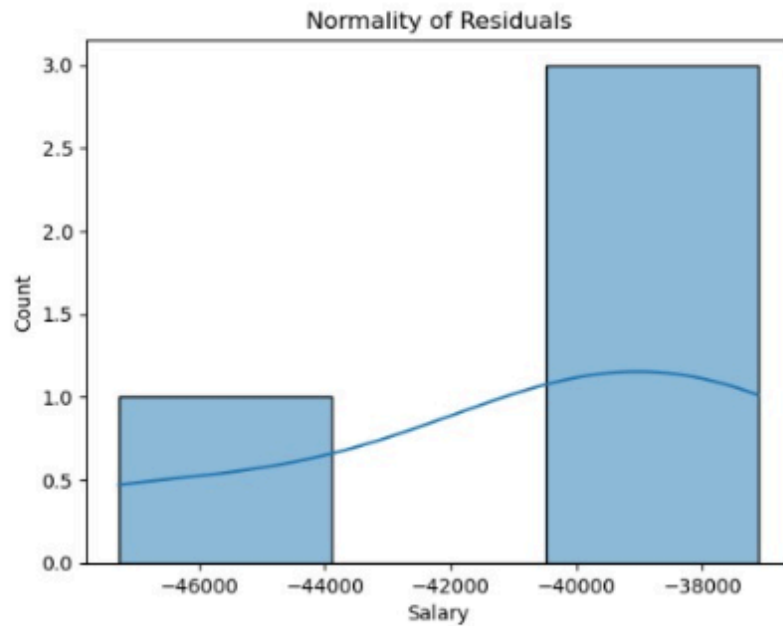
```
plt.show()
```

Output:

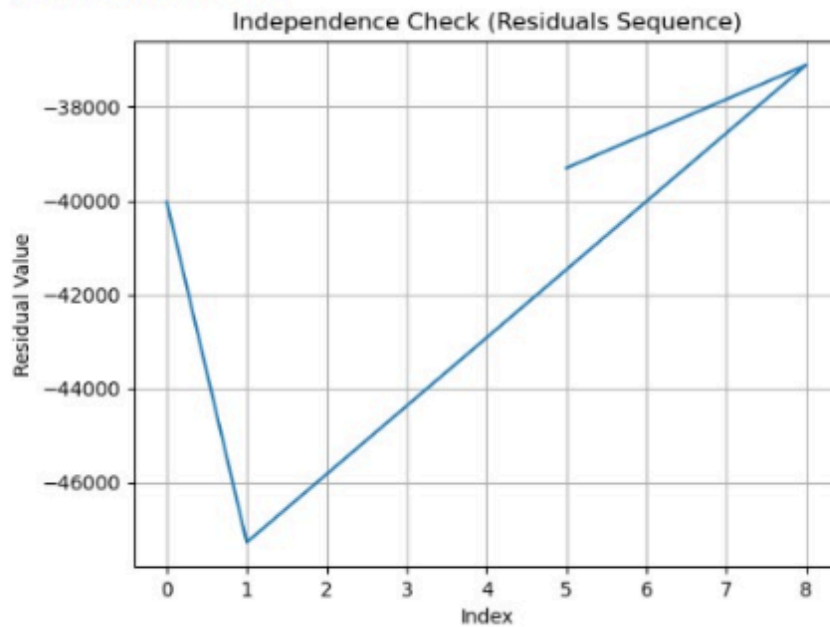


Homoscedasticity Check





Shapiro-Wilk Test:
 Statistic: 0.8660272359848022
 p-value: 0.2823839783668518



15. Write a python code to apply KNN Classification Problem

Ans:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
```

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")

# ----- Encode Target Variable (Gender) -----
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender']) # Male=1, Female=0

# ----- Select Features and Target -----
X = df[['Weight', 'Pincode', 'Salary']] # independent variables
y = df['Gender'] # dependent variable

# ----- Train-Test Split -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----- KNN Model -----
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# ----- Predictions -----
y_pred = knn.predict(X_test)

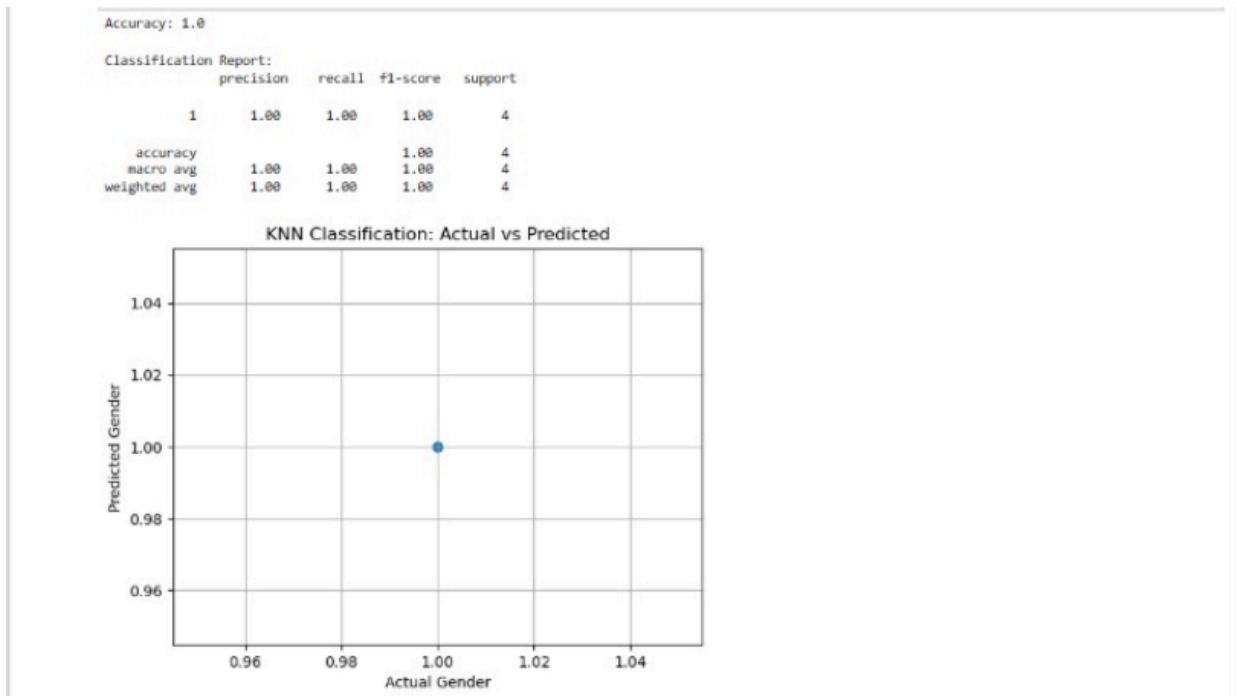
# ----- Evaluation -----
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# ----- Visualization -----
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Gender")
plt.ylabel("Predicted Gender")
plt.title("KNN Classification: Actual vs Predicted")
plt.grid(True)

```

plt.show()

Output:



16. Write a python code to apply KNN Regression Problem?

Ans:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")

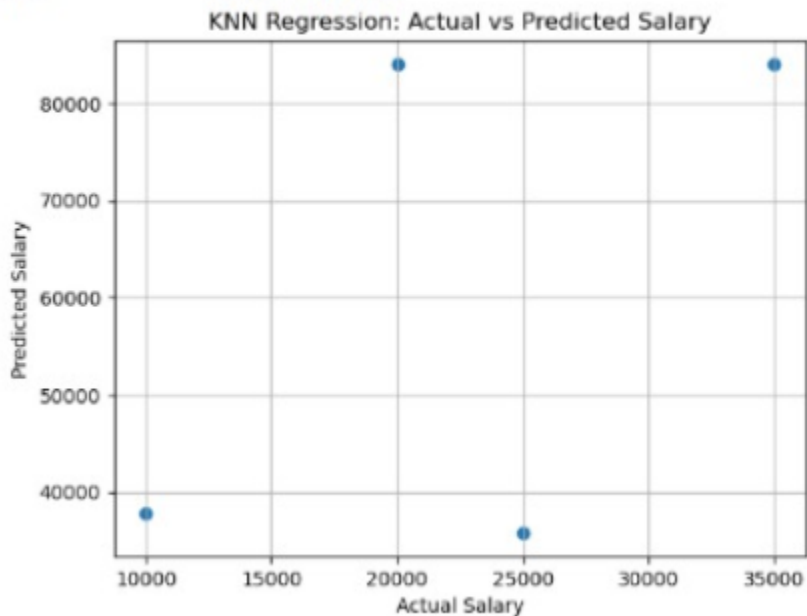
# ----- Select Features and Target -----
X = df[["Weight", "Pincode"]] # independent variables
y = df["Salary"]             # dependent variable

# ----- Train-Test Split -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

```
)  
# ----- KNN Regression Model -----  
knn_reg = KNeighborsRegressor(n_neighbors=5)  
knn_reg.fit(X_train, y_train)  
# ----- Predictions -----  
y_pred = knn_reg.predict(X_test)  
# ----- Evaluation -----  
mae = mean_absolute_error(y_test, y_pred)  
mse = mean_squared_error(y_test, y_pred)  
rmse = np.sqrt(mse)  
print("Mean Absolute Error :", mae)  
print("Mean Squared Error :", mse)  
print("RMSE          :", rmse)  
# ----- Visualization -----  
plt.scatter(y_test, y_pred)  
plt.xlabel("Actual Salary")  
plt.ylabel("Predicted Salary")  
plt.title("KNN Regression: Actual vs Predicted Salary")  
plt.grid(True)  
plt.show()
```


Output:

```
Mean Absolute Error : 37900.0  
Mean Squared Error : 1846620000.0  
RMSE : 42972.316670154054
```



17. Write a python code to apply SVM Classification Problem.

Ans:

```
import pandas as pd  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import LabelEncoder, StandardScaler  
from sklearn.svm import SVC  
from sklearn.metrics import accuracy_score, classification_report  
import matplotlib.pyplot as plt  
# Load dataset  
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")  
# ----- Encode Target Variable (Gender) -----  
le = LabelEncoder()  
df['Gender'] = le.fit_transform(df['Gender']) # Male=1, Female=0  
# ----- Select Features and Target -----  
X = df[['Weight', 'Pincode', 'Salary']]  
y = df['Gender']  
# ----- Feature Scaling (Important for SVM) -----  
scaler = StandardScaler()  
X = scaler.fit_transform(X)  
# ----- Train-Test Split -----
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# ----- SVM Model -----
svm_clf = SVC(kernel='rbf') # rbf kernel (best for nonlinear data)
svm_clf.fit(X_train, y_train)
# ----- Predictions -----
y_pred = svm_clf.predict(X_test)
# ----- Evaluation -----
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
# ----- Visualization -----
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Gender")
plt.ylabel("Predicted Gender")
plt.title("SVM Classification: Actual vs Predicted")
plt.grid(True)
plt.show()

```

Output:

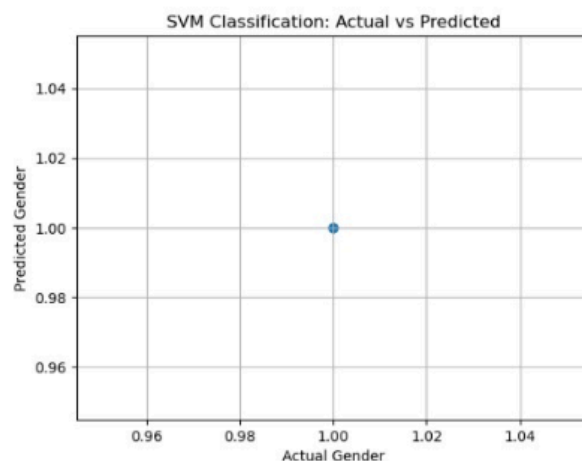
```

Accuracy: 1.0
Classification Report:
      precision    recall  f1-score   support

     1         1.00      1.00      1.00         4

 accuracy          1.00          1.00          1.00         4
 macro avg          1.00          1.00          1.00         4
 weighted avg          1.00          1.00          1.00         4

```



18. Write a python code to apply SVM Regression Problem.

Ans: import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVR

from sklearn.metrics import mean_absolute_error, mean_squared_error

import numpy as np

import matplotlib.pyplot as plt

Load dataset

df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")

----- Select Features and Target -----

X = df[['Weight', 'Pincode']]

y = df['Salary']

----- Feature Scaling (Very important for SVR) -----

scaler_x = StandardScaler()

scaler_y = StandardScaler()

X_scaled = scaler_x.fit_transform(X)

y_scaled = scaler_y.fit_transform(y.values.reshape(-1, 1)).ravel()

----- Train-Test Split -----

X_train, X_test, y_train, y_test = train_test_split(
 X_scaled, y_scaled, test_size=0.2, random_state=42
)

----- SVR Model -----

svr = SVR(kernel='rbf')

svr.fit(X_train, y_train)

----- Predictions -----

y_pred_scaled = svr.predict(X_test)

Inverse scaling to get original salary values

y_pred = scaler_y.inverse_transform(y_pred_scaled.reshape(-1, 1)).ravel()

y_test_orig = scaler_y.inverse_transform(y_test.reshape(-1, 1)).ravel()

----- Evaluation -----

mae = mean_absolute_error(y_test_orig, y_pred)

mse = mean_squared_error(y_test_orig, y_pred)

rmse = np.sqrt(mse)

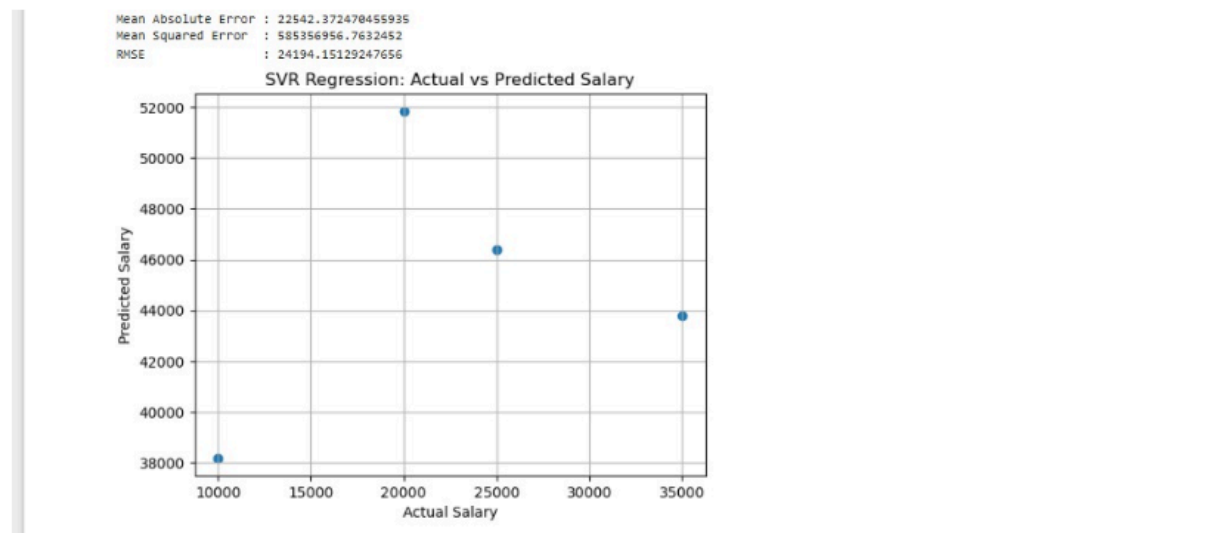
print("Mean Absolute Error :", mae)

print("Mean Squared Error :", mse)

print("RMSE :", rmse)

```
# ----- Visualization -----
plt.scatter(y_test_orig, y_pred)
plt.xlabel("Actual Salary")
plt.ylabel("Predicted Salary")
plt.title("SVR Regression: Actual vs Predicted Salary")
plt.grid(True)
plt.show()
```

Output:



19. Write a python code to apply Decision Tree Classification Problem.

Ans:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")

# ----- Encode Target (Gender) -----
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender']) # Male=1, Female=0

# ----- Select Features and Target -----
```

```

X = df[['Weight', 'Pincode', 'Salary']]
y = df['Gender']
# ----- Train-Test Split -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# ----- Decision Tree Model -----
dt_clf = DecisionTreeClassifier(criterion='entropy', random_state=42)
dt_clf.fit(X_train, y_train)
# ----- Predictions -----
y_pred = dt_clf.predict(X_test)
# ----- Evaluation -----
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
# ----- Visualization of Decision Tree -----
plt.figure(figsize=(12, 8))
plot_tree(dt_clf, filled=True, feature_names=X.columns, class_names=['Female', 'Male'])
plt.title("Decision Tree Classification")
plt.show()
Output:

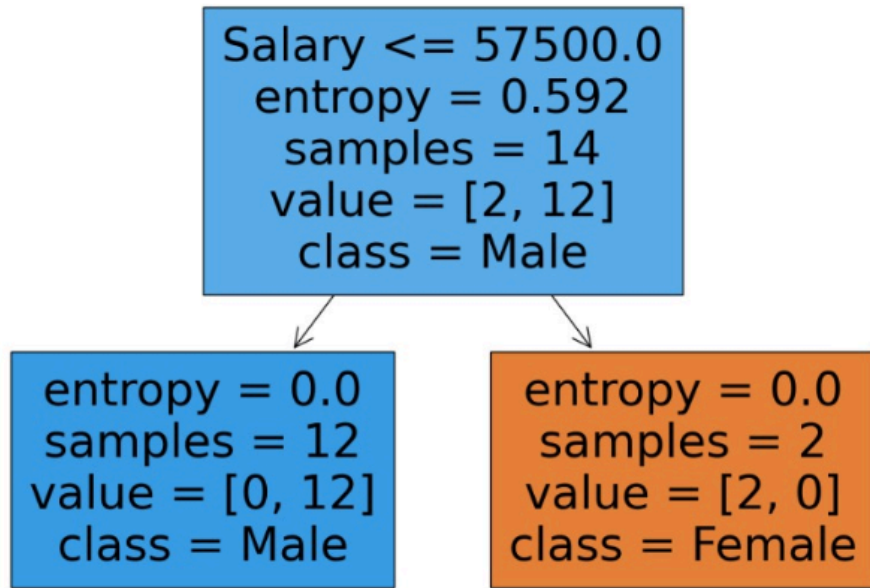
```

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
1	1.00	1.00	1.00	4
accuracy			1.00	4
macro avg	1.00	1.00	1.00	4
weighted avg	1.00	1.00	1.00	4

Decision Tree Classification



20. Write a python code to apply the Decision Tree Regression Problem.

Ans:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")

# ----- Select Features and Target -----
X = df[["Weight", "Pincode", "Salary"]] # independent variables
y = df["Salary"]                       # dependent variable
  
```

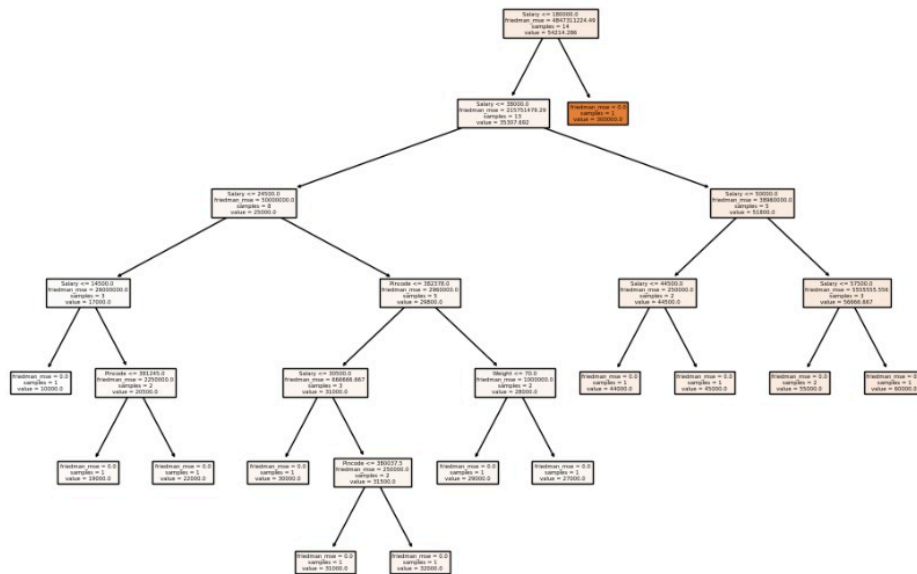
```

# ----- Train-Test Split -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# ----- Decision Tree Regression Model -----
dt_reg = DecisionTreeRegressor(criterion='friedman_mse', random_state=42)
dt_reg.fit(X_train, y_train)
# ----- Predictions -----
y_pred = dt_reg.predict(X_test)
# ----- Evaluation -----
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print("Mean Absolute Error :", mae)
print("Mean Squared Error :", mse)
print("RMSE          :", rmse)
# ----- Visualization -----
plt.figure(figsize=(12, 8))
plot_tree(dt_reg, filled=True, feature_names=X.columns)
plt.title("Decision Tree Regression")
plt.show()
# Scatter plot (Actual vs Predicted)
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Salary")
plt.ylabel("Predicted Salary")
plt.title("Decision Tree Regression: Actual vs Predicted Salary")
plt.grid(True)
plt.show
Output:

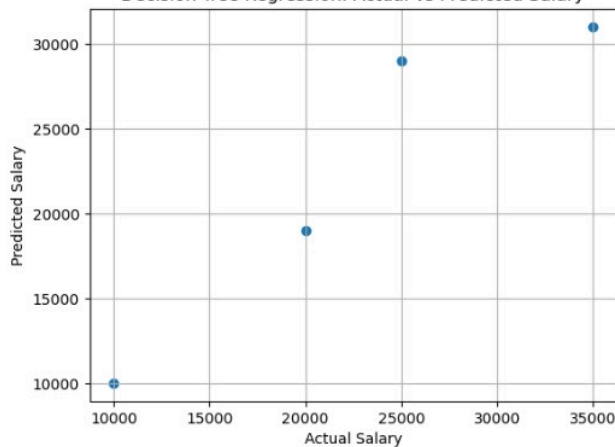
```

Mean Absolute Error : 2250.0
Mean Squared Error : 8250000.0
RMSE : 2872.281323269014

Decision Tree Regression



Decision Tree Regression: Actual vs Predicted Salary



21. Write a python code to apply Random Forest Classification Problem.

Ans: import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report

import matplotlib.pyplot as plt

Load dataset


```

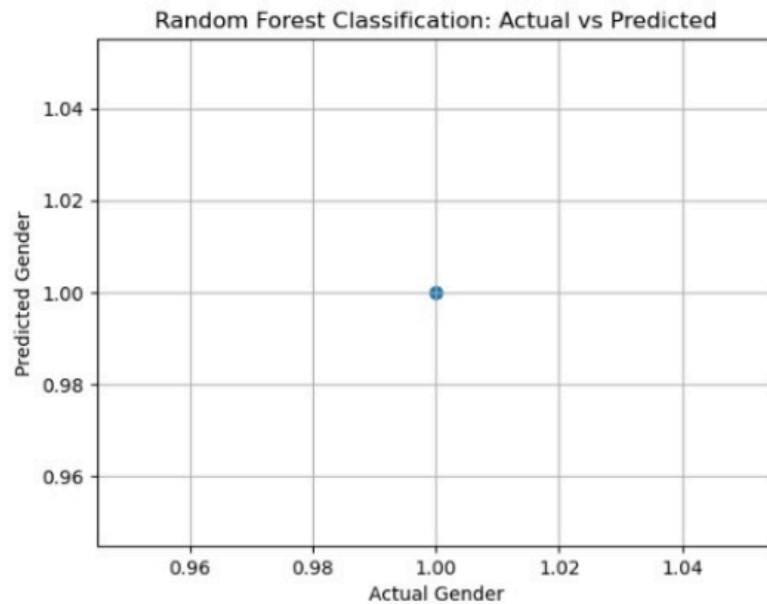
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")
# ----- Encode Target (Gender) -----
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender']) # Male=1, Female=0
# ----- Select Features and Target -----
X = df[['Weight', 'Pincode', 'Salary']]
y = df['Gender']
# ----- Train-Test Split -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# ----- Random Forest Classifier -----
rf_clf = RandomForestClassifier(
    n_estimators=100,
    criterion='gini',
    random_state=42
)
rf_clf.fit(X_train, y_train)
# ----- Predictions -----
y_pred = rf_clf.predict(X_test)
# ----- Evaluation -----
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
# ----- Visualization (Actual vs Predicted) -----
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Gender")
plt.ylabel("Predicted Gender")
plt.title("Random Forest Classification: Actual vs Predicted")
plt.grid(True)
plt.show()
Output:

```

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
1	1.00	1.00	1.00	4
accuracy			1.00	4
macro avg	1.00	1.00	1.00	4
weighted avg	1.00	1.00	1.00	4



22. Write a python code to apply Random Forest Regression Problem ?

Ans:

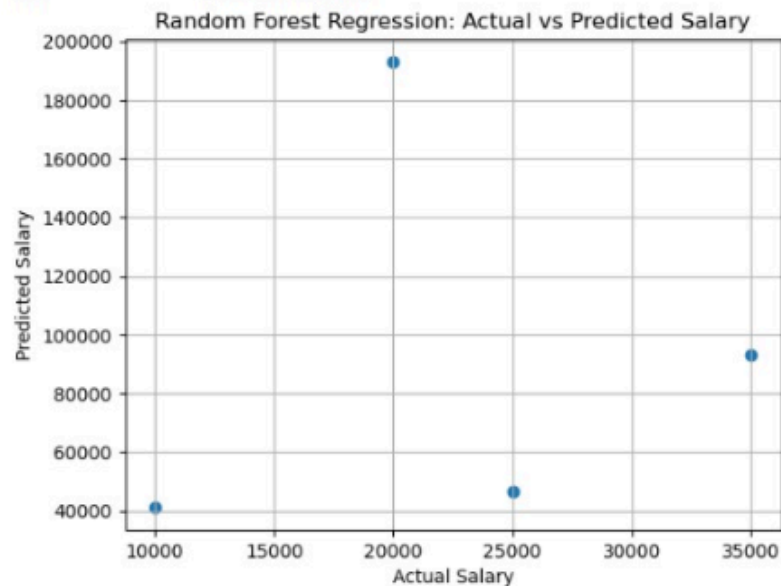
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
import matplotlib.pyplot as plt
# Load dataset
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")
# ----- Select Features and Target -----
X = df[['Weight', 'Pincode']]
y = df['Salary']
# ----- Train-Test Split -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
)
# ----- Random Forest Regression Model -----
rf_reg = RandomForestRegressor(
    n_estimators=100,
    criterion='friedman_mse',
    random_state=42
)
rf_reg.fit(X_train, y_train)
# ----- Predictions -----
y_pred = rf_reg.predict(X_test)
# ----- Evaluation -----
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print("Mean Absolute Error :", mae)
print("Mean Squared Error :", mse)
print("RMSE          :", rmse)
# ----- Visualization -----
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Salary")
plt.ylabel("Predicted Salary")
plt.title("Random Forest Regression: Actual vs Predicted Salary")
plt.grid(True)
plt.show()
```

Output:

Output:

```
Mean Absolute Error : 71037.5
Mean Squared Error : 8691496625.0
RMSE : 93228.1965126431
```



23. Write a python code to apply Logistic Regression

Ans: import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, classification_report

import matplotlib.pyplot as plt

Load dataset

df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")

----- Encode Target (Gender) -----

le = LabelEncoder()

df['Gender'] = le.fit_transform(df['Gender']) # Male=1, Female=0

----- Select Features and Target -----

X = df[['Weight', 'Pincode', 'Salary']]

y = df['Gender']

----- Feature Scaling (important for logistic regression) -----

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

----- Train-Test Split -----

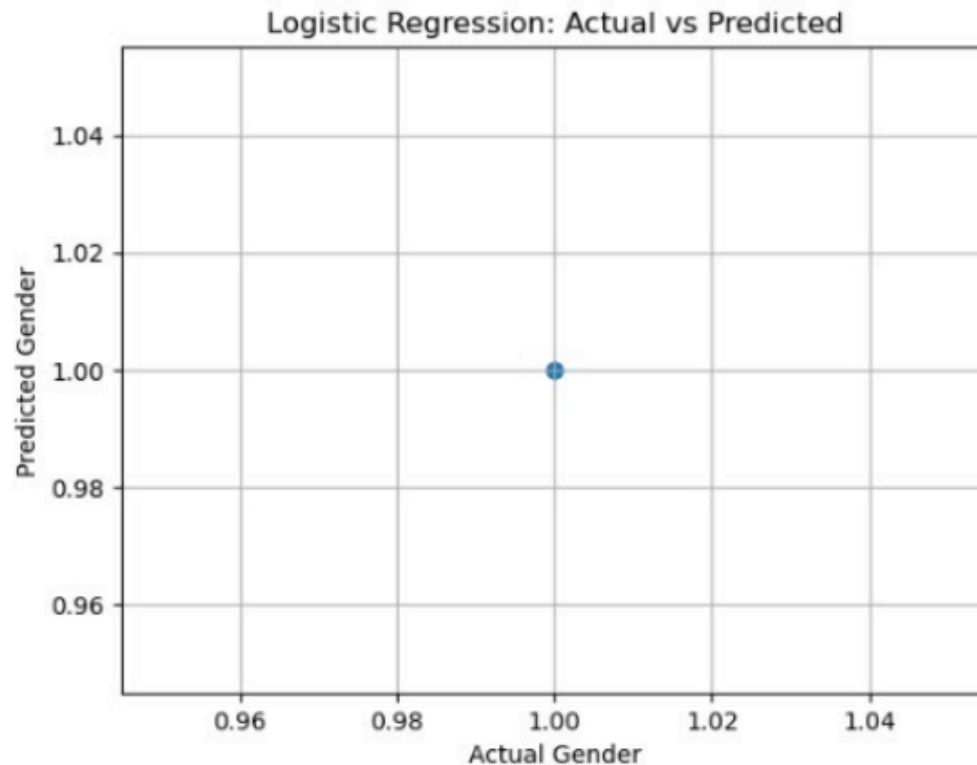
```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
# ----- Logistic Regression Model -----
log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)
# ----- Predictions -----
y_pred = log_reg.predict(X_test)
# ----- Evaluation -----
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
# ----- Visualization -----
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Gender")
plt.ylabel("Predicted Gender")
plt.title("Logistic Regression: Actual vs Predicted")
plt.grid(True)
plt.show()
```

Output:

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
1	1.00	1.00	1.00	4
accuracy			1.00	4
macro avg	1.00	1.00	1.00	4
weighted avg	1.00	1.00	1.00	4



24. Write a python code to apply Naïve Bayes Classifier.

Ans:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
# Load dataset
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")
# ----- Encode Target (Gender) -----
```

```

le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender']) # Male=1, Female=0
# ----- Select Features and Target -----
X = df[['Weight', 'Pincode', 'Salary']]
y = df['Gender']
# ----- Feature Scaling (Recommended for Naive Bayes) -----
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# ----- Train-Test Split -----
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
# ----- Naïve Bayes Model -----
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
# ----- Predictions -----
y_pred = nb_model.predict(X_test)
# ----- Evaluation -----
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
# ----- Visualization -----
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Gender")
plt.ylabel("Predicted Gender")
plt.title("Naive Bayes Classification: Actual vs Predicted")
plt.grid(True)
plt.show()

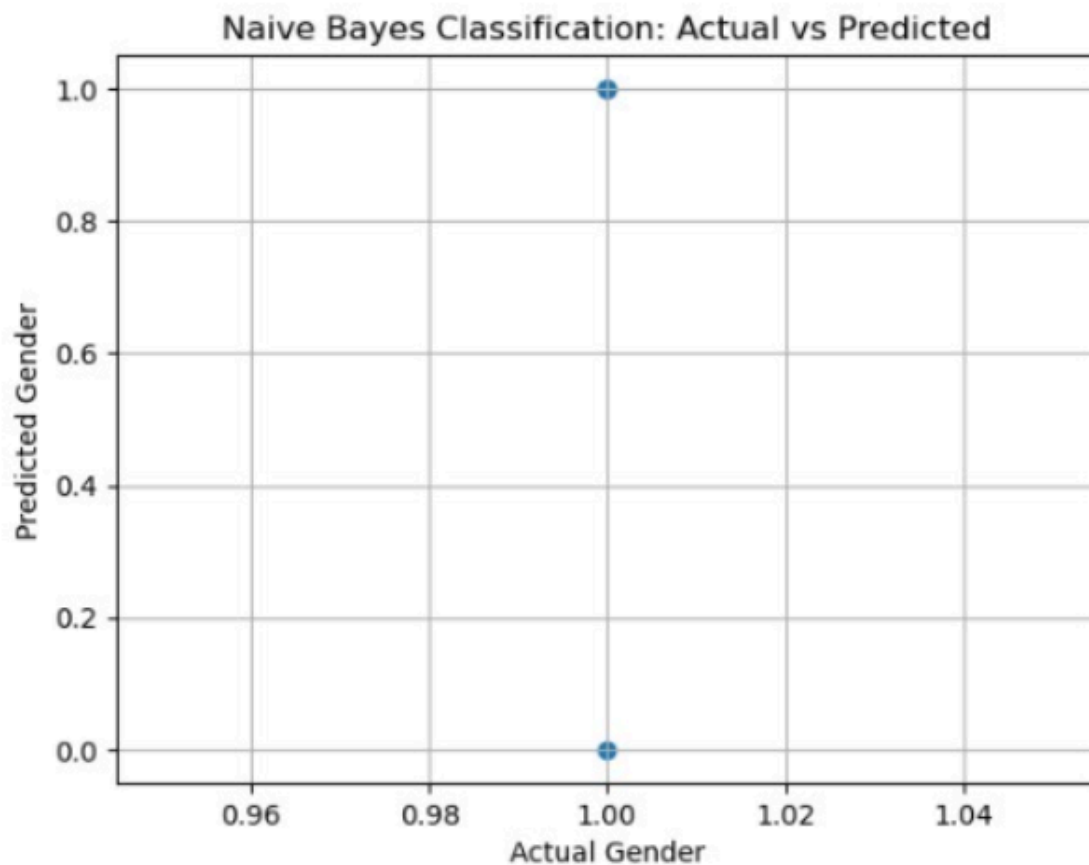
```

Output:

Accuracy: 0.75

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	0
1	1.00	0.75	0.86	4
accuracy			0.75	4
macro avg	0.50	0.38	0.43	4
weighted avg	1.00	0.75	0.86	4



25. Write a python code to apply Regression Metrics.

Ans: import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

Load dataset


```

df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")
# ----- Select Features and Target -----
X = df[['Weight', 'Pincode']]
y = df['Salary']
# ----- Train-Test Split -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# ----- Train Regression Model -----
model = LinearRegression()
model.fit(X_train, y_train)
# ----- Predictions -----
y_pred = model.predict(X_test)
# ----- Regression Metrics -----
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("R2 Score:", r2)

```

Output:

```

Mean Absolute Error (MAE): 51063.454220696025
Mean Squared Error (MSE): 2684886031.8371344
Root Mean Squared Error (RMSE): 51815.88590227069
R2 Score: -32.044751161072426

```

26. Write a python code to apply Confusion metrics, Accuracy, Precision, F1- Score, Recall.

Ans:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

```

```

from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score,
f1_score
from sklearn.linear_model import LogisticRegression
import seaborn as sns
import matplotlib.pyplot as plt
# Load dataset
df = pd.read_excel("myDataset (1).xlsx", engine="openpyxl")
# ----- Encode Target (Gender) -----
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender']) # Male=1, Female=0
# ----- Select Features & Target -----
X = df[['Weight', 'Pincode', 'Salary']]
y = df['Gender']
# ----- Feature Scaling -----
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# ----- Train-Test Split -----
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
# ----- Logistic Regression Model -----
model = LogisticRegression()
model.fit(X_train, y_train)
# ----- Predictions -----
y_pred = model.predict(X_test)
# ----- Metrics -----
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print("Confusion Matrix:\n", cm)
print("\nAccuracy :", accuracy)
print("Precision:", precision)
print("Recall :", recall)
print("F1 Score :", f1)
# ----- Visualization of Confusion Matrix -----

```

```
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Female', 'Male'],
            yticklabels=['Female', 'Male'])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

Output:

